

SeEDM Platform: Comprehensive Project Documentation

Project Name: SeEDM (Secure Enterprise Data Masking & Migration)

Version: 1.0

Date: March 31, 2026

1. Executive Summary & Overview

1.1 The Business Problem: Data Friction

In modern software development, a critical bottleneck known as “Data Friction” exists. Development and QA teams require realistic, production-like data to test applications effectively. However, stringent privacy regulations like GDPR, HIPAA, and CCPA strictly prohibit copying raw Personally Identifiable Information (PII) to non-production environments. Traditional solutions—manual SQL scripts or legacy ETL tools—are slow, error-prone, and expensive, leading to delays and the use of poor-quality synthetic data.

1.2 The SeEDM Solution

SeEDM is a purpose-built infrastructure platform that automates the secure streaming of data from production to non-production environments. It leverages Java Virtual Threads and Spring Boot to move data while applying **real-time, in-memory data protection** (Masking, Mocking, Hashing). This ensures sensitive data never lands on disk in plain text, reducing data provisioning time from days to minutes.

Key Value Propositions:

- **Compliance:** Zero PII leakage via deterministic, in-memory masking.
- **Performance:** 10x faster than traditional ETL using parallel virtual threads.
- **Integrity:** Preserves complex database relationships (Referential Integrity) even after encryption.

Key Capabilities:

- **Multi-Source Support:** Handles **Relational SQL** (PostgreSQL, Oracle, DB2), **NoSQL** (MongoDB), and **JSON Files** (directory-to-directory).
 - **Workflow-Driven:** A consistent 6-step workflow for each migration type, as shown in the UI screenshots.
 - **Advanced Data Protection:** Implements **Mock Data** (Faker), **Partial Masking** (e.g., credit card redaction), and **Format-Preserving Hashing** to maintain referential integrity.
-

2. System Architecture

The platform follows a **Hub-and-Spoke** architecture designed for high throughput and security isolation.

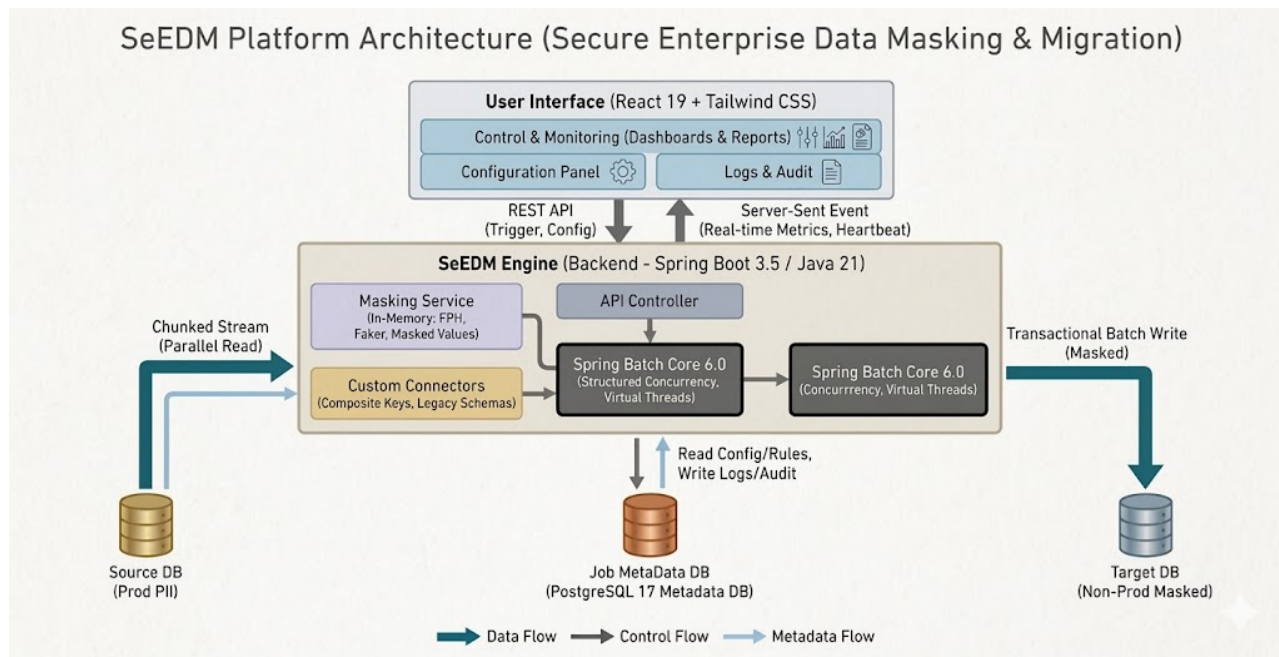
2.1 Component Breakdown

- **The SeEDM Engine (Backend):** The core orchestration layer built with **Spring Boot 4** and **Spring Batch 6**. It manages the “Extract-Mask-Load” pipeline using structured concurrency. It reads raw data, applies masking rules in the JVM heap, and commits to the target.

- **The Cockpit (Frontend):** A React 19-based dashboard that visualizes throughput, progress, and health via Server-Sent Events (SSE). It is designed as a “blind” dashboard that never receives or displays actual business data.
- **The Vault (Metadata DB):** An internal PostgreSQL database that stores Job Profiles, Masking Rules, Audit Logs, and Checkpoints for restartability.
- **Connectors:** Specialized JDBC components for relational databases, MongoDB drivers, and File I/O handlers for JSON processing.

2.2 Architecture Diagram

The system flow is as follows:



3. Data Flow & Workflow Details

Each migration type (Relational, NoSQL, JSON) follows a consistent 6-stage workflow, mirroring the UI screenshots provided.

3.1 Workflow Stages

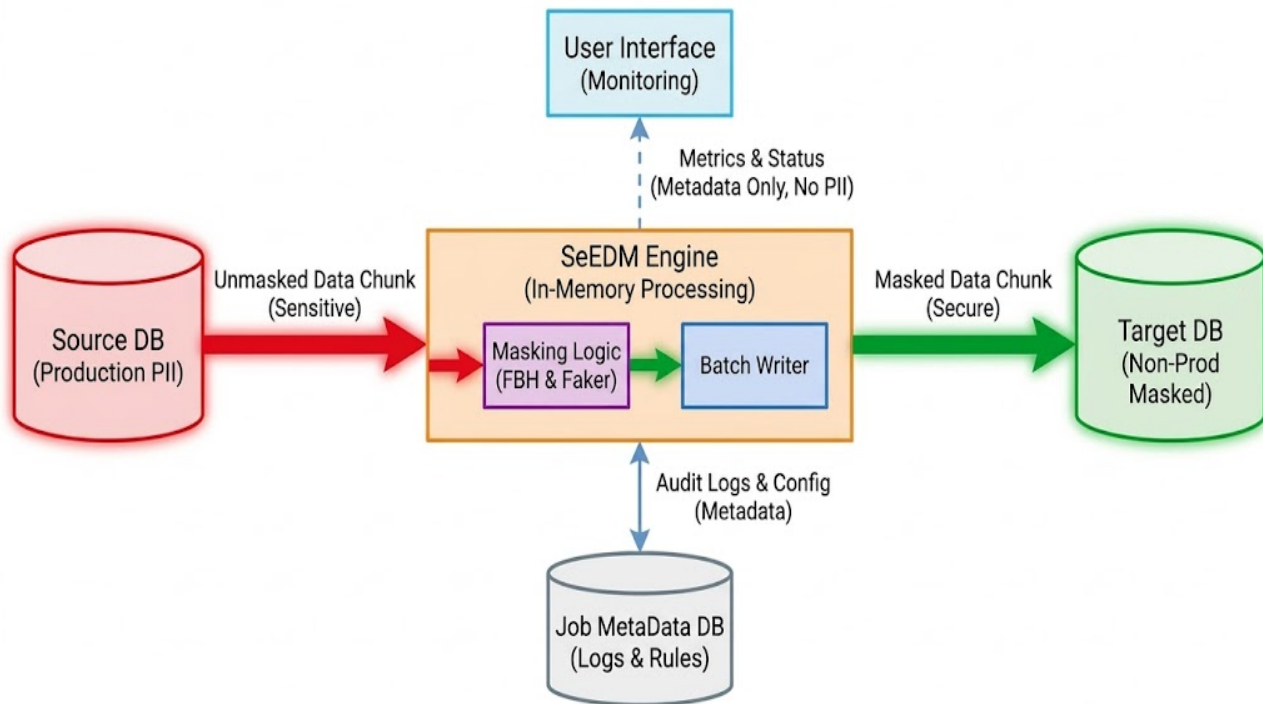
1. **Connection:** User defines source and destination connections (JDBC URLs, credentials, directories).
2. **Tables / Scope:** User selects specific tables (for SQL) or scans directories (for JSON) to include in the migration.
3. **Rules:** User assigns masking rules (e.g., Standard Fake Data, Format-Preserving Hashing, Redaction) to specific columns or JSON fields.
4. **Review:** The system displays a summary of configuration, source/destination details, and the total scope of the job for user confirmation.
5. **Launch:** User initiates the migration job.

6. **Monitoring:** The UI redirects to the “Active Job Status” screen to monitor throughput, table-level progress, and logs.

3.2 Data Flow Diagram

The following diagram illustrates the chunk-oriented processing flow for relational databases:

SeEDM Platform Data Flow: Secure & Masked Migration



4. Core Technical Implementation

4.1 Spring Batch with Chunk-Oriented Processing

The platform uses **Spring Batch 6** to manage the data pipeline. The chunk-oriented style processes data in configurable batches (e.g., 1000 rows). This keeps memory usage constant regardless of the total dataset size.

4.2 Error Handling & Restart Options

- **Intelligent Resume:** Checkpoints are saved in the **Vault** (PostgreSQL) after every successful chunk. If a job fails (e.g., due to network failure), it resumes from the last saved checkpoint, not from the beginning.
- **Fault Tolerance:** The `SkipPolicy` allows the job to skip a configurable number of invalid records (e.g., data type mismatch) without failing the entire job, while logging them for audit.

4.3 Parallel Processing

To achieve high performance, SeEDM uses **Java 21 Virtual Threads**. It automatically detects large tables and partitions them, processing each partition in parallel.

5. Data Masking Techniques

The platform implements three primary data protection techniques to satisfy different security and operational requirements.

5.1 Mocking Data (Faker)

This technique replaces sensitive data with realistic but fictitious data.

- **Use Case:** Replacing real names or emails in a non-production environment.
- **Implementation:** The Java `Faker` library generates contextually relevant data.
- **Example:**
 - **Input:** `john.doe@realcompany.com`
 - **Output:** `alice.smith@example.com`
 - **Rule:** `FAKER_EMAIL`

5.2 Partial Masking (Redaction)

This technique reveals only a portion of the data, which is useful for support or debugging scenarios.

- **Use Case:** Masking credit card numbers or phone numbers while keeping a referenceable part.
- **Implementation:** String manipulation logic that replaces characters with `x` or `*`.
- **Example:**
 - **Input:** `4532-1234-5678-9876`
 - **Output:** `XXXX-XXXX-XXXX-9876`
 - **Rule:** `REDACTION (KEEP LAST 4)`

5.3 Format-Preserving Hashing Technique

This is the most critical technique for maintaining **Referential Integrity**. It applies a deterministic keyed hash to the input while preserving the original data format (e.g., a 5-digit integer remains a 5-digit integer).

- **Use Case:** Hashing `user_id` or `account_number` columns that are used as foreign keys in other tables. By hashing them **deterministically**, the relationship between tables remains intact.
- **Implementation:** The system uses a keyed hash function (e.g., HMAC) and then formats the output to match the original column's type and length. No encryption is involved; it is a one-way transformation that is repeatable for the same input and key.
- **Example:** Maintaining Table Relationships
 - **Table A (Customer):**
 - **Column id (PK):** `12345` → `58291`

- **Table B (Orders):**
 - Column `customer_id` (FK): 12345 → 58291
- **Outcome:** Even after hashing, `Orders` still correctly reference their parent `Customer` records because the same input value always yields the same hashed output.

6. Technology Stack & Requirements

6.1 Software Requirements

Category	Technology	Version
Framework	Spring Boot	3.5.0
Language	Java (OpenJDK)	21 (LTS)
Batch Core	Spring Batch	6.x
Frontend	React	19
Security	Spring Security	7 (OAuth2 / RBAC)
Build Tool	Maven	3.9+
Database Driver	PostgreSQL JDBC, MongoDB Driver Latest	
Container	Docker, Kubernetes	Latest

6.2 Hardware Requirements (for Production Node)

- **Processor:** 8 vCPU (to maximize parallel thread utilization)
- **RAM:** 32 GB (to support high-concurrency in-memory processing buffers)
- **Network:** 10 Gbps (to minimize latency between source and target)

7. User Interface Workflow (from Screenshots)

The UI is designed to guide the user through the configuration and monitoring process:

1. **Login** (`login.html`): Secured entry point to the platform.
2. **Select Migration Type** (`select-db.html`): The hub where users choose between **Relational SQL**, **NoSQL Document**, or **JSON Files**.
3. **Configure Migration** (`configure-migration.html`): The main workflow page for relational databases.
 - *Step 1:* Enter Source/Destination JDBC URLs, credentials, and schemas.

- *Step 2:* Review and select specific tables for migration.
- *Step 3:* Assign masking rules per column (STD, ENC, PRT, etc.).
- *Step 4:* Review the full configuration before launching.

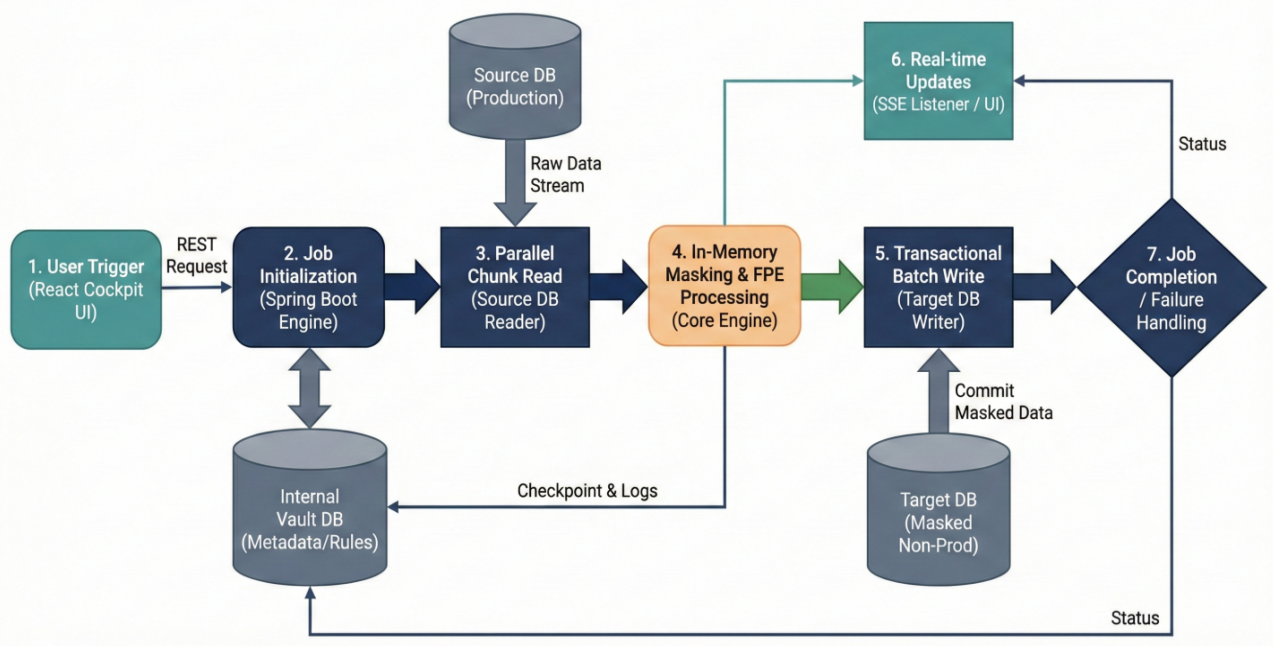
4. **JSON Migration (workflow-json.html):** A similar workflow but tailored for JSON files.

- User specifies source and destination directories.
- The system scans for available JSON files and fields.
- Masking rules are applied via drag-and-drop of fields onto rule tiles.

5. **Active Job Status (active-job.html):** The monitoring screen.

- Displays real-time **Migration Throughput** (Rows per Second).
- Shows a **Table Execution Log** with read/write counts, status, and progress.
- Shows the **Elapsed Time** and **Job ID**.

SeEDM Platform Migration Lifecycle Flow



8. Legacy System & Complex Data Handling

The design explicitly addresses “brownfield” environments common in enterprises:

- **Virtual Foreign Keys:** Allows users to define implicit relationships (e.g., `Table_A.Cust_Ref` to `Table_B.Customer_ID`) so the engine can process dependent tables in the correct order, even if database constraints are missing.
- **Composite Key Support:** Implements a custom `PagingQueryProvider` that can generate stable pagination SQL for tables without a single primary key (e.g., `ORDER BY Branch_ID, Sequence_No`).

- **LOB/CLOB Streaming:** Uses zero-copy buffers to handle large BLOB (images) and CLOB (XML/Text) columns efficiently without exhausting heap memory.

9. Operation

Monitoring (The Cockpit)

The User Interface is designed for **Observability**, not Data Browsing.

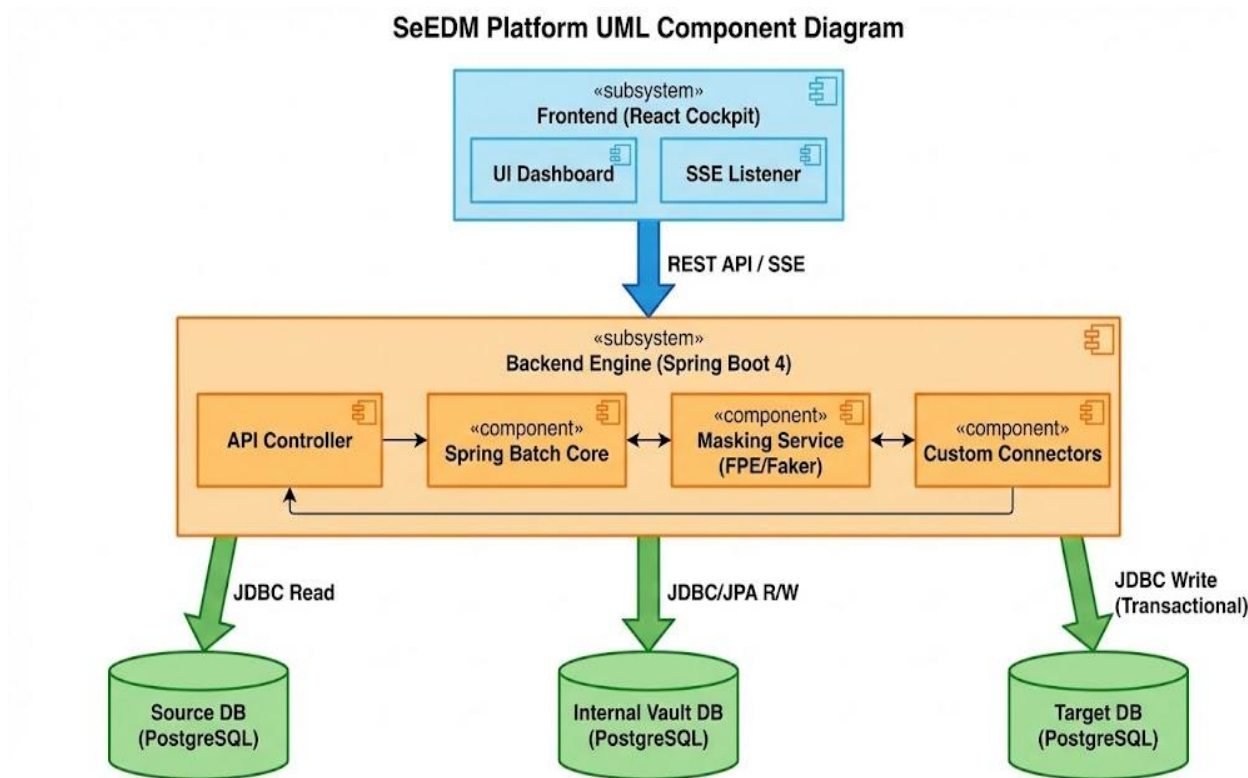
7.1 Blind Dashboard Features

al

- **Throughput Speedometer:** Visualizes Rows Per Second (RPS) in real-time.
- **Partition Heatmap:** Shows which parallel threads are active, blocked, or completed.
- **Error Fingerprinting:** Aggregates errors by type (e.g., "Data Truncation: 45 errors") rather than showing sensitive row data.

7.2 Real-Time Architecture

- **Protocol: Server-Sent Events (SSE).**
- **Flow:** The Spring Boot backend pushes lightweight JSON metrics packets every 500ms. The React Frontend listens to this stream and updates the DOM efficiently without polling.



9. Conclusion

The SeEDM Platform transforms data migration from a high-risk manual task into a secure, automated, and observable infrastructure service. By combining a consistent user workflow with advanced technical features like Spring Batch chunking, intelligent restartability, and robust data protection techniques (Faker, Redaction, Format-Preserving Hashing), it ensures data privacy, operational integrity, and high performance across relational, NoSQL, and file-based data sources.

SeEDM Platform – Demo Presentation

Slide 1: Title Slide

- **Title:** SeEDM – Secure Enterprise Data Masking & Migration Platform
 - **Subtitle:** Accelerate Development with Production-Safe Data
 - **Date:** March 2026
 - **Presenter:** [Your Name]
 - **Company/Logo**
-

Slide 2: The Challenge – Data Friction

- **Headline:** Developers need real data, but privacy laws forbid raw PII
 - **Key Pain Points:**
 - Manual SQL scripts & legacy ETL are slow, error-prone, expensive
 - Data provisioning takes days → development bottlenecks
 - Poor synthetic data hides bugs
 - **Regulatory Pressure:** GDPR, HIPAA, CCPA, DPDP
-

Slide 3: SeEDM – The Solution

- **Headline:** Automate Secure Data Movement from Production to Non-Prod
 - **Three Pillars:**
 - **Compliance:** Zero PII leakage – in-memory masking
 - **Performance:** 10× faster using Java virtual threads
 - **Integrity:** Preserve referential relationships with Format-Preserving Hashing
 - **Key Capabilities:**
 - Multi-source: Relational (PostgreSQL, Oracle, DB2), MongoDB, JSON files
 - Consistent 6-step workflow
 - Real-time monitoring dashboard
-

Slide 4: High-Level Architecture

- **Diagram:** Hub-and-Spoke Architecture
 - **Components:**

- **React Cockpit (Frontend):** Blind dashboard, SSE updates
 - **SeEDM Engine (Spring Boot 4 + Spring Batch 6):** Orchestration & masking
 - **Vault DB (PostgreSQL):** Metadata, checkpoints, audit logs
 - **Connectors:** JDBC, MongoDB, File I/O
- **Flow:** User → Cockpit → Engine → Extract → Mask → Load → Target
-

Slide 5: The 6-Step Workflow

- **Visual:** 6-step numbered flow
 1. **Connection** – Source & destination details
 2. **Tables / Scope** – Select tables or scan JSON directories
 3. **Rules** – Assign masking per column/field
 4. **Review** – Confirm configuration summary
 5. **Launch** – Start migration job
 6. **Monitoring** – Live throughput & table logs
-

Slide 6: Technical Deep Dive – Spring Batch & Resilience

- **Chunk-Oriented Processing:** Batches of 1000 rows → constant memory
 - **Intelligent Resume:** Checkpoint after every chunk → resume from failure, not from start
 - **Parallel Processing:** Virtual threads auto-partition large tables
 - **Fault Tolerance:** Skip invalid records, log errors, continue
-

Slide 7: Data Protection Techniques

- **Three Masking Methods:**

Technique	Description	Example
Mocking (Faker)	Replace with realistic fake data	john@real.com → alice@example.com
Partial Masking (Redaction)	Show only last 4 digits	4532-1234-5678-9876 → XXXX- XXXX-XXXX-9876

Format-Preserving Hashing

Deterministic keyed hash, preserves format & referential integrity $12345 \rightarrow 58291$ (same for FK across tables)

- **Why Format-Preserving Hashing?** Maintains foreign key relationships without encryption
-

Slide 8: Maintaining Referential Integrity (Example)

- **Scenario:** Two tables with foreign key relationship
 - **Customer** (PK `id` = 12345)
 - **Orders** (FK `customer_id` = 12345)
 - **After Format-Preserving Hashing:**
 - `Customer.id` $\rightarrow$ 58291
 - `Orders.customer_id` $\rightarrow$ 58291
 - **Result:** Relationship preserved, data anonymized
-

Slide 9: UI Walkthrough – Screenshots

- **Login Screen** – Secure entry
- **Select Migration Type** – Relational / NoSQL / JSON
- **Configure Migration** – Connection, Tables, Rules, Review
- **JSON Migration** – Directory selection, field mapping
- **Active Job Status** – Throughput chart, table execution log, elapsed time

Place actual screenshots from your application here

Slide 10: Legacy & Complex Data Handling

- **Virtual Foreign Keys:** Define implicit relationships for legacy DBs without constraints
 - **Composite Key Support:** Stable pagination for tables without a single PK
 - **LOB/CLOB Streaming:** Zero-copy handling of large binary/text data
-

Slide 11: Operational Monitoring – The Cockpit

- **Blind Dashboard Features:**
 - Real-time throughput (Rows per Second)
 - Partition heatmap – active/blocked threads
 - Error fingerprinting – aggregated error types
 - **Real-time Architecture:**
 - Server-Sent Events (SSE) push every 500ms
 - No polling – lightweight, efficient updates
-

Slide 12: Summary & Value Proposition

- **SeEDM transforms data migration into a secure, automated service**
- **Key Takeaways:**
 - Provision production-like data in minutes, not days
 - Ensure compliance with deterministic, in-memory masking
 - Preserve data integrity with Format-Preserving Hashing
 - Scale effortlessly with virtual threads and parallel processing
- **Next Steps:** Schedule a live demo / pilot

